Mrunal Samrutwar P10 Extra Spotify

CS23O82

Aim: To perform Hierarchical Clustering using agglomerative methods and interpret dendrograms for cluster formation.

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
path="/content/drive/MyDrive/ML datasets/Spotify_Songs_Dataset.csv"
df = pd.read_csv(path)
df.head()
```

| | Unnamed: 0 | track_id | artists | album_name | track_name | popularity |
|---|---|---|---|---|---|---|
| **0** | 0 | 5SuOikwiRyPMVoIQDJUgSV | Gen Hoshino | Comedy | Comedy | 73 |
| **1** | 1 | 4qPNDBW1i3p13qLCt0Ki3A | Ben Woodward | Ghost (Acoustic) | Ghost - Acoustic | 55 |
| **2** | 2 | 1iJBSr7s7jYXzM8EGcbK5b | Ingrid Michaelson;ZAYN | To Begin Again | To Begin Again | 57 |
| **3** | 3 | 6lfxq3CG4xtTiEg7opyCyx | Kina Grannis | Crazy Rich Asians (Original Motion Picture Sou... | Can't Help Falling In Love | 71 |
| **4** | 4 | 5vjLSffimiIP26QG5WcN2K | Chord Overstreet | Hold On | Hold On | 82 |

5 rows × 21 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 114000 entries, 0 to 113999
Data columns (total 21 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
```

```
 0   Unnamed: 0        114000 non-null  int64
 1   track_id          114000 non-null  object
 2   artists           113999 non-null  object
 3   album_name        113999 non-null  object
 4   track_name        113999 non-null  object
 5   popularity        114000 non-null  int64
 6   duration_ms       114000 non-null  int64
 7   explicit          114000 non-null  bool
 8   danceability      114000 non-null  float64
 9   energy            114000 non-null  float64
 10  key               114000 non-null  int64
 11  loudness          114000 non-null  float64
 12  mode              114000 non-null  int64
 13  speechiness       114000 non-null  float64
 14  acousticness      114000 non-null  float64
 15  instrumentalness  114000 non-null  float64
 16  liveness          114000 non-null  float64
 17  valence           114000 non-null  float64
 18  tempo             114000 non-null  float64
 19  time_signature    114000 non-null  int64
 20  track_genre       114000 non-null  object
dtypes: bool(1), float64(9), int64(6), object(5)
memory usage: 17.5+ MB
```

```
df.describe()
```

| | Unnamed: 0 | popularity | duration_ms | danceability | energy | ke |
|---|---|---|---|---|---|---|
| **count** | 114000.000000 | 114000.000000 | 1.140000e+05 | 114000.000000 | 114000.000000 | 114000.00000 |
| **mean** | 56999.500000 | 33.238535 | 2.280292e+05 | 0.566800 | 0.641383 | 5.30914 |
| **std** | 32909.109681 | 22.305078 | 1.072977e+05 | 0.173542 | 0.251529 | 3.55998 |
| **min** | 0.000000 | 0.000000 | 0.000000e+00 | 0.000000 | 0.000000 | 0.00000 |
| **25%** | 28499.750000 | 17.000000 | 1.740660e+05 | 0.456000 | 0.472000 | 2.00000 |
| **50%** | 56999.500000 | 35.000000 | 2.129060e+05 | 0.580000 | 0.685000 | 5.00000 |
| **75%** | 85499.250000 | 50.000000 | 2.615060e+05 | 0.695000 | 0.854000 | 8.00000 |
| **max** | 113999.000000 | 100.000000 | 5.237295e+06 | 0.985000 | 1.000000 | 11.00000 |

```
df.shape
df.columns
```

```
Index(['Unnamed: 0', 'track_id', 'artists', 'album_name', 'track_name',
       'popularity', 'duration_ms', 'explicit', 'danceability', 'energy',
       'key', 'loudness', 'mode', 'speechiness', 'acousticness',
       'instrumentalness', 'liveness', 'valence', 'tempo', 'time_signature',
       'track_genre'],
      dtype='object')
```

```
df.isnull().sum()
```

| | 0 |
|---|---|
| Unnamed: 0 | 0 |
| track_id | 0 |
| artists | 1 |
| album_name | 1 |
| track_name | 1 |
| popularity | 0 |
| duration_ms | 0 |
| explicit | 0 |
| danceability | 0 |
| energy | 0 |
| key | 0 |
| loudness | 0 |
| mode | 0 |
| speechiness | 0 |
| acousticness | 0 |
| instrumentalness | 0 |
| liveness | 0 |
| valence | 0 |
| tempo | 0 |
| time_signature | 0 |
| track_genre | 0 |

**dtype:** int64

```
df = df.dropna()
```

```
df = df.drop(['track_id','track_name','artist_name'], axis=1, errors='ignore')
```

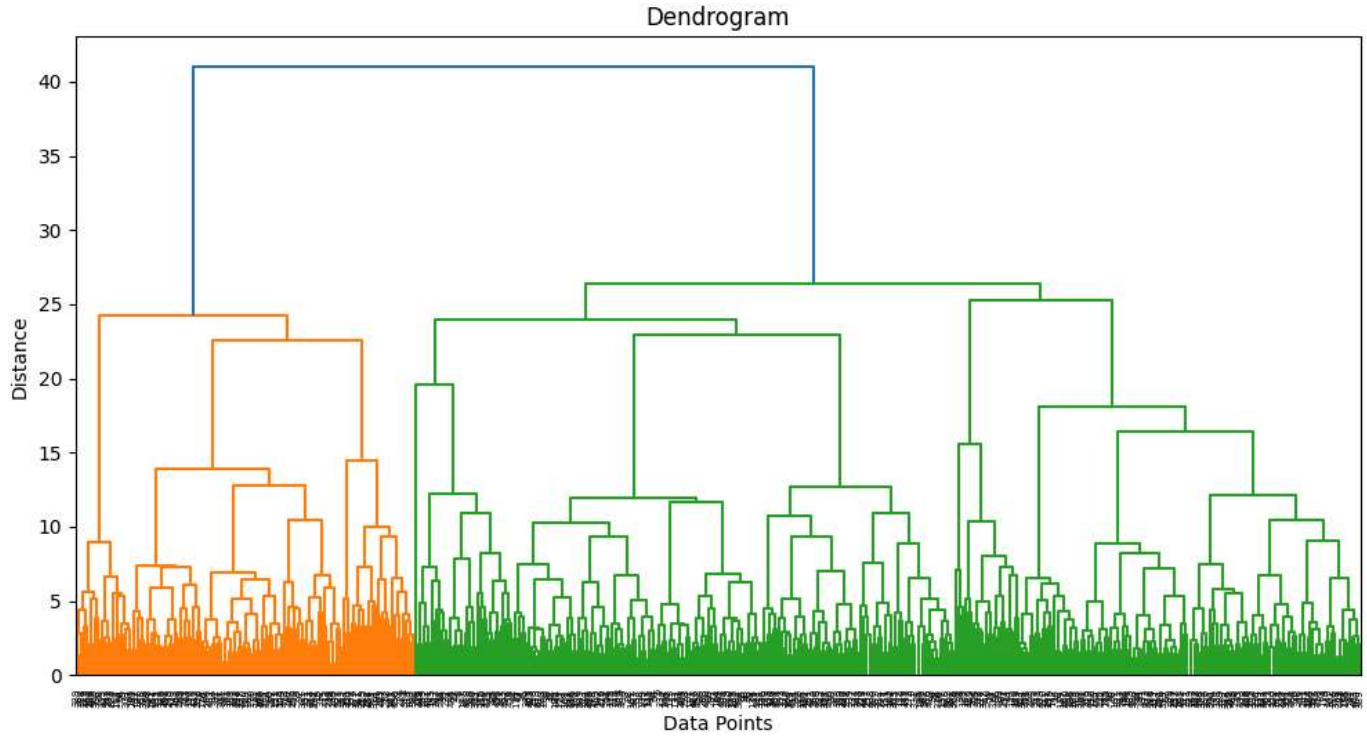
```
df = df.select_dtypes(include=['int64','float64'])
```

```
df = df.sample(n=500, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

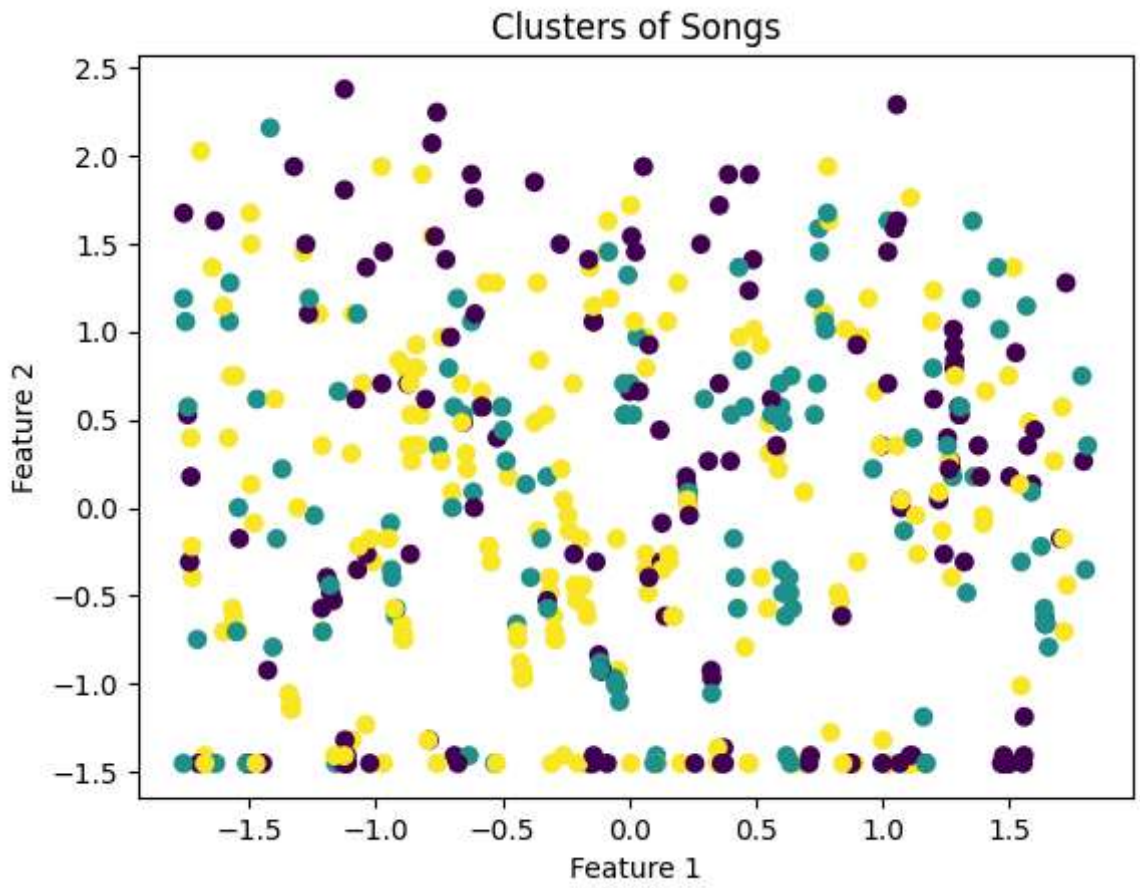
```
scaled_data = scaler.fit_transform(df)
```

```
import scipy.cluster.hierarchy as sch
plt.figure(figsize=(12,6))
sch.dendrogram(sch.linkage(scaled_data, method='ward'))
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```




```
from sklearn.cluster import AgglomerativeClustering
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
y_pred = hc.fit_predict(scaled_data)
print(y_pred)
```

```
[1 2 1 0 2 2 0 2 2 2 1 2 1 0 1 0 0 2 2 0 0 0 2 0 2 0 0 2 1 2 1 0 2 1 2 2 2
 2 0 1 1 2 2 1 0 0 0 2 2 2 2 2 0 2 2 2 2 0 2 2 1 0 1 1 2 2 0 2 1 0 1 2 2
 0 1 2 2 2 1 2 1 0 0 0 2 2 1 2 2 1 1 1 0 2 1 0 0 2 2 0 0 2 0 1 1 2 0 1 2 2
 1 2 1 0 2 2 2 1 0 1 0 2 2 1 2 0 0 0 2 2 2 2 2 0 2 2 2 0 0 0 1 1 2 2 2 2 0
 0 0 0 0 1 2 1 2 0 2 1 2 1 2 1 0 2 2 2 0 2 0 1 2 2 2 1 2 0 2 2 2 0 1 0 1 0
 0 0 1 1 0 2 1 0 1 0 0 2 0 1 2 2 0 2 1 2 0 0 0 1 2 0 0 0 1 1 0 2 0 1 1 2 0
```

```
2 2 0 1 1 2 2 0 2 0 1 1 2 2 1 2 0 2 2 0 0 0 2 1 2 2 0 2 1 2 1 1 1 1 2 2 2
0 0 2 1 0 2 0 1 2 2 1 1 1 2 2 1 0 2 0 0 2 1 1 2 0 1 0 0 2 0 0 2 1 2 1 0 2
0 0 2 0 0 1 1 1 2 1 0 2 0 1 1 0 1 0 0 2 2 0 0 2 2 2 0 1 0 2 1 0 0 1 2 2 2
0 1 2 2 1 2 0 0 2 1 2 2 0 2 1 0 2 2 1 0 2 1 0 2 2 1 1 1 1 2 2 0 2 1 1 2 0
2 1 2 0 2 1 1 0 0 0 0 1 1 2 2 2 0 2 2 0 1 2 1 1 1 2 0 0 2 2 0 2 2 0 2 2 2
2 0 1 2 2 1 2 0 1 0 0 2 0 1 2 0 2 1 2 0 2 2 2 1 2 1 0 2 2 2 2 2 0 1 2 1 0
0 2 0 0 2 1 1 2 2 0 1 1 2 0 2 2 0 0 2 0 0 1 0 1 1 1 2 2 0 0 0 0 1 2 1 0 2
1 2 0 0 2 2 1 0 2 2 0 0 1 2 1 2 1 1 2]
```

```
plt.scatter(scaled_data[:,0], scaled_data[:,1], c=y_pred)
plt.title("Clusters of Songs")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```